GRANDE ECOLE

LA MANU

Création ■ Management ■ Numérique

L’algorithmie

A retenir !

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Si vous ne retenez qu'**une seule chose** de cette présentation, ce serait **réfléchir avant d'agir !**

Introduction

Algorithmie & Pseudo code

Concepts fondamentaux & pensée algorithmique

Qu'est-ce qu'un algorithme ?

Définition formelle : Une suite finie, univoque (non ambiguë) et ordonnée d'instructions ou d'opérations permettant de résoudre un problème donné ou d'obtenir un résultat précis à partir d'éléments d'entrée.

En d'autres termes :

Un algorithme, c'est une méthode **pas à pas** pour résoudre un **problème**.

C'est la description exacte du comment on passe d'une situation initiale A à une situation finale B.

Anatomie universelle d’une tâche

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Tout système logique (qu'il s'agisse d'un objet connecté, d'un parcours sur un écran, d'un formulaire ou d'un programme) fonctionne selon la même **triade fondamentale** :

Composante	Rôle fonctionnel
Entrées (Inputs)	Ce que la machine/le système reçoit pour travailler.
Traitement (Process)	La transformation de la donnée, l'application des règles.
Sortie (Outputs)	Le résultat restitué à l'utilisateur ou à un autre système.

Exemple

Le distributeur de boissons

Entrées :

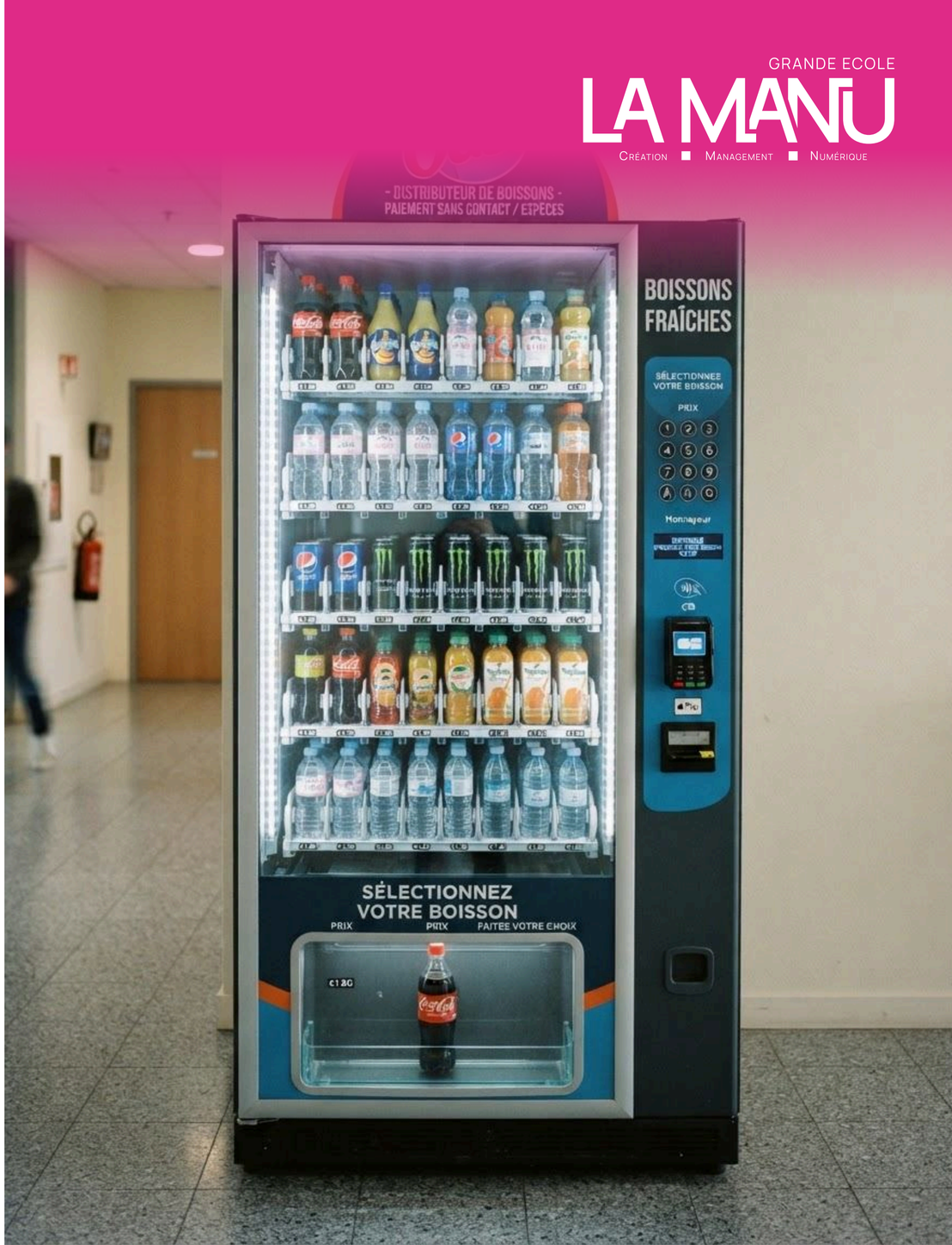
- Pièce insérée (5€)
- Touche appuyée (Soda)

Traitement :

- Vérifier si la somme versée est supérieure ou égale au prix (2€)
- Calculer la monnaie à rendre
- Déclencher la préparation de la boisson

Sorties :

- Donner la bouteille, et rendre la monnaie



L’importance de la précision

Les 3 pièges classiques

GRANDE ECOLE LA MANU
Création ■ Management ■ Numérique

L'implicite du contexte 1

Humain : « Fais de l’eau chaude. » → L'humain prend une casserole ou une bouilloire, met de l’eau et chauffe.

Machine : Sans préciser le récipient et la source d'eau, elle ne sait pas où trouver l’eau ni où la chauffer.

L'ordre des opérations 2

Humain : Enfile tes chaussettes et tes chaussures. (Même si l'ordre est inversé à l'oral, l'humain met d'abord les chaussettes).

Machine : Si l'instruction dit "Mets les chaussures puis mets les chaussettes", elle essaiera de mettre les chaussettes par-dessus les chaussures.

L'Ambiguïté du langage naturel 3

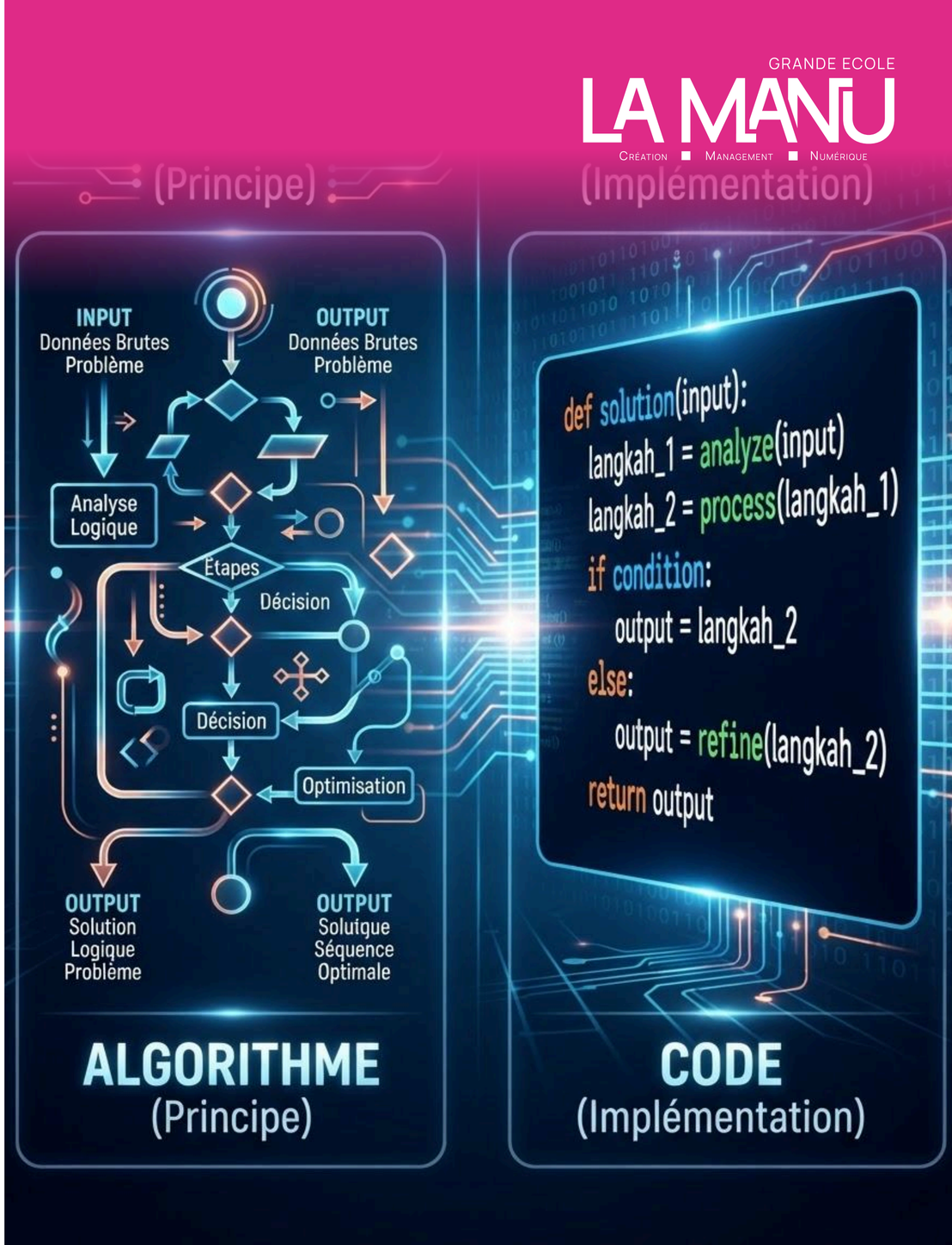
Phrase : « Il faut couper la viande avec le couteau sur la table. »

Incertitude logique : Faut-il couper la viande qui se trouve sur la table, ou utiliser le couteau qui se trouve sur la table ?

Algorithme vs Code : Réflexion vs Traduction

Il est essentiel de séparer la **conception** de la **réalisation**.

Étape	Rôle	Analogie
1. Problème / Besoins	Comprendre ce qu'on cherche à résoudre	Le cahier des charges d'un projet
2. Algorithme	Concevoir la solution pas-à-pas (Logique pure, universelle)	Le plan de construction / Le schéma logique
3. Code / Programmation	Traduire la solution dans un langage informatique précis	La construction effective selon les règles locales



A vous !

L'Exercice du Robot Humanoïde

CHALLENGE : Écrire l'algorithme d'un dessin géométrique

Par **binôme**, sur une feuille : Écrivez la suite d'instructions étape par étape pour qu'un robot dessine une maison au tableau (un carré, un toit triangulaire).

Seule règle : Vous n'avez pas le droit de parler pendant que le robot dessine, il lit juste votre feuille.

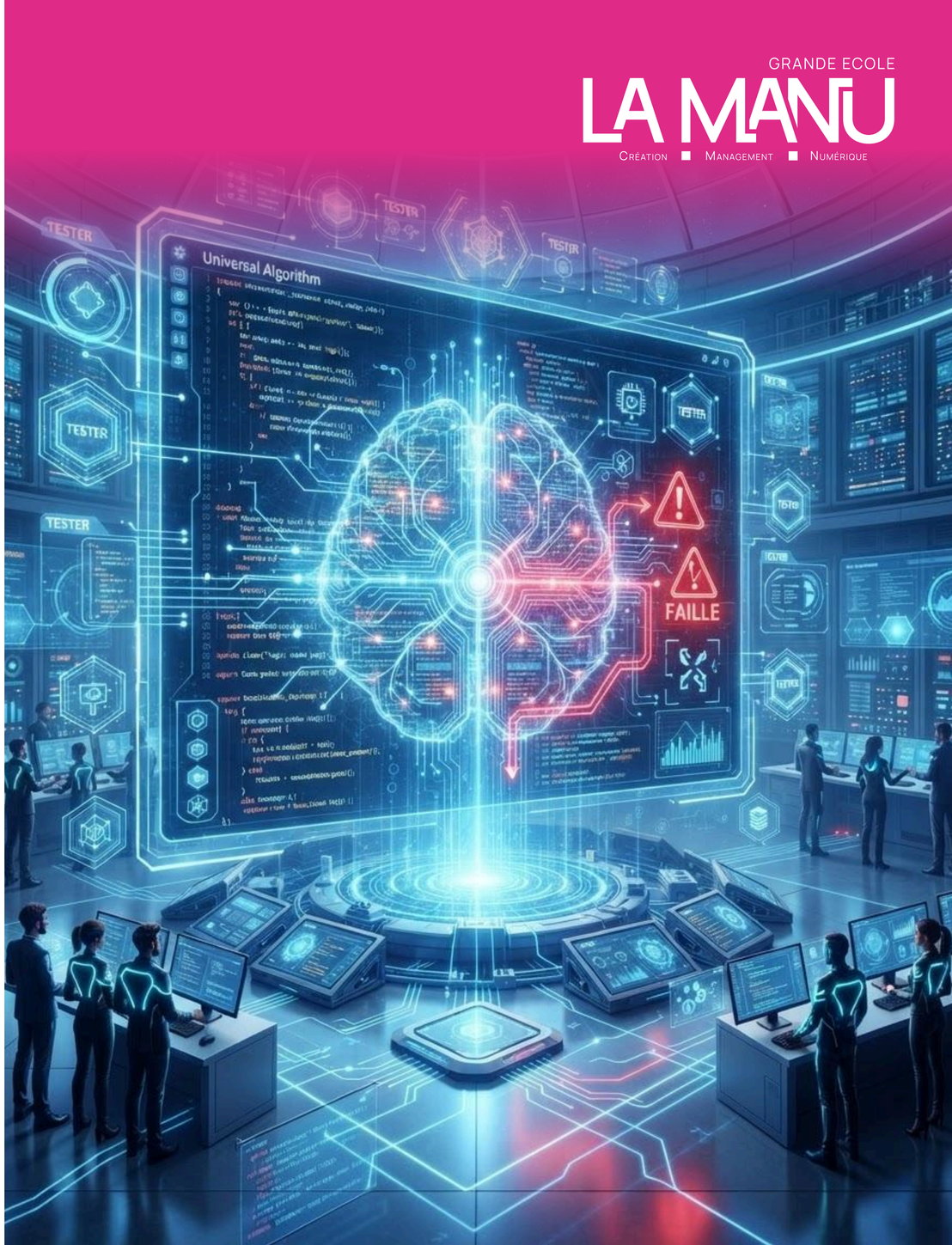
Algorithme valide

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

```
1  // --- INITIALISATION ---
2  1. Placer le feutre au centre du tableau.
3  2. Poser le feutre sur le tableau.
4  // --- TRACÉ DU CARRE (MURS) ---
5  3. Tracer une ligne de 40 cm vers la DROITE.
6  4. Tracer une ligne de 40 cm vers le HAUT.
7  5. Tracer une ligne de 40 cm vers la GAUCHE.
8  6. Tracer une ligne de 40 cm vers le BAS.
9  (Remarque : Le feutre est revenu au point d'origine)
10 // --- DEPLACEMENT VERS LE TOIT ---
11 7. Lever le feutre du tableau.
12 8. Remonter le feutre de 40 cm vers le HAUT (au coin supérieur gauche).
13 9. Poser le feutre sur le tableau.
14 // --- TRACÉ DU TOIT ---
15 10. Tracer une ligne de 28 cm à 45° (vers le HAUT et la DROITE).
16 11. Tracer une ligne de 28 cm à 45° (vers le BAS et la DROITE).
17
18 // --- FIN ---
19 12. Lever le feutre du tableau.
```

A retenir !

Un algorithme est universel	Un algorithme ne tolère aucun implicite	Tester, c'est trouver des failles
Il ne dépend d'aucun ordinateur ni d'aucun langage.	Tout doit être explicite (Début, Actions séquentielles, Fin).	Déceler les cas non prévus est le cœur du travail d'analyse.



Pseudo-code

Variables et affectations

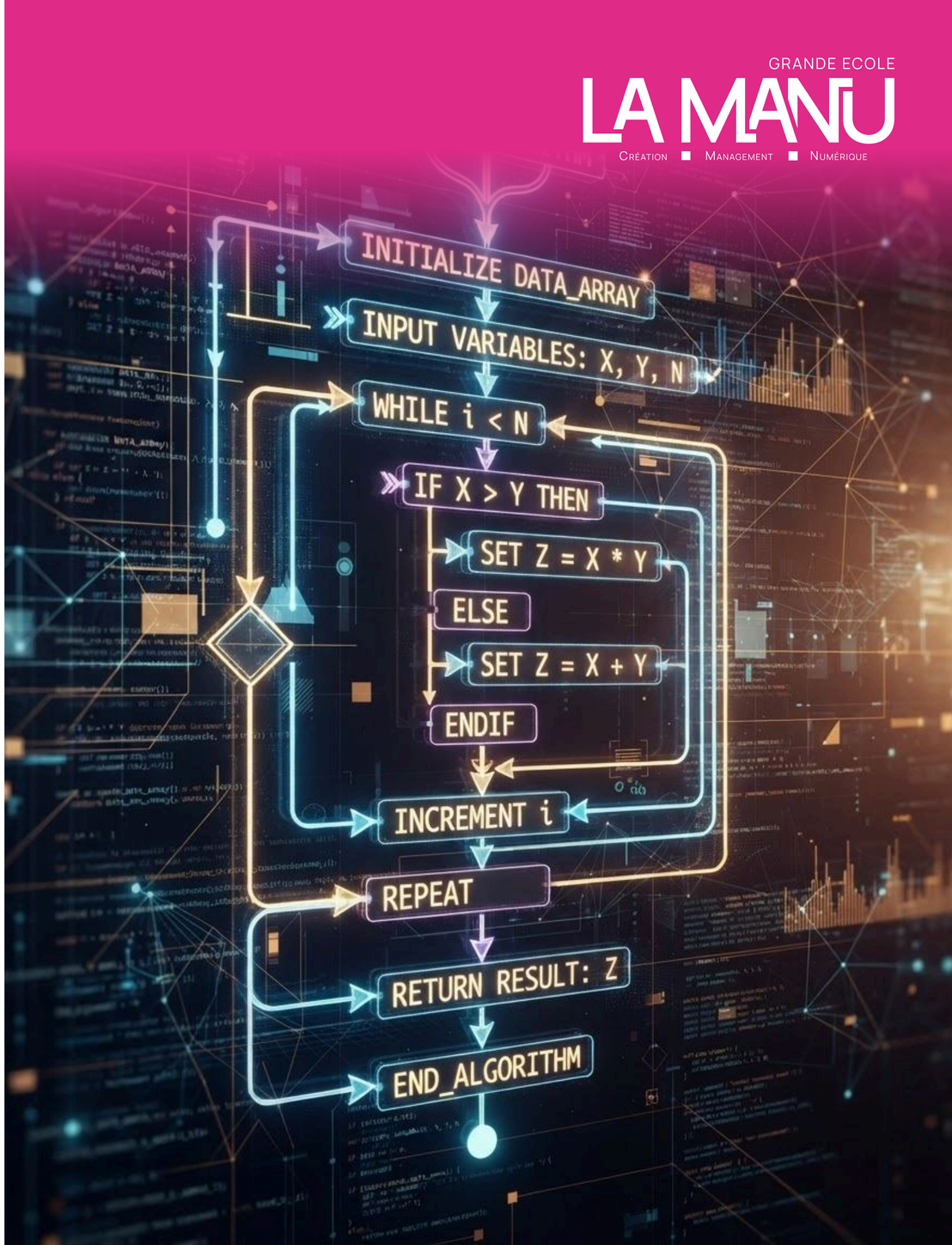
Normaliser l'écriture

Le **pseudo-code** est un langage de description algorithmique **formel** mais **non exécutable** directement par une machine.

Il permet de **concevoir** et de **valider** la logique d'un programme en français structuré, en s'affranchissant des **règles syntaxiques** d'un langage de programmation spécifique.

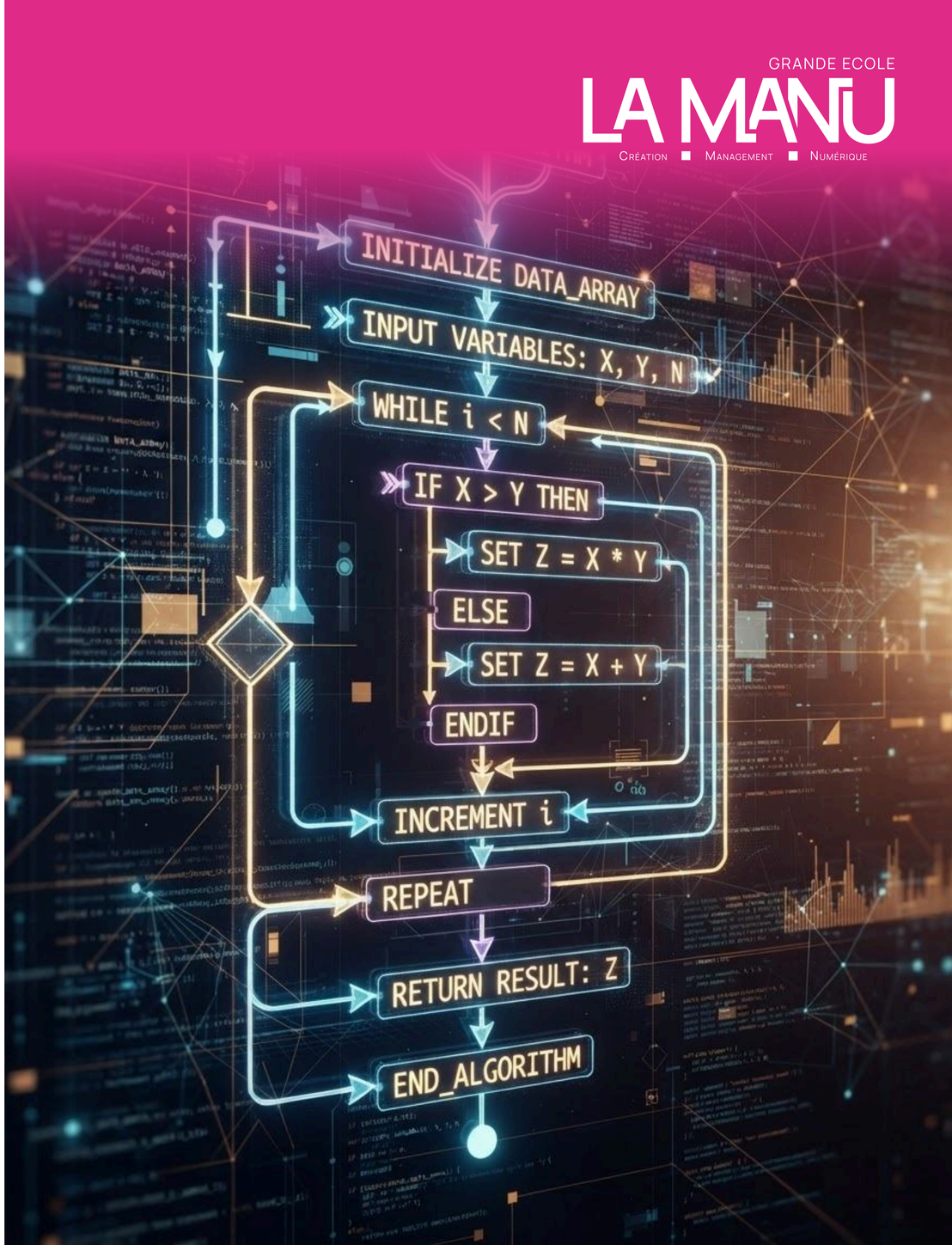
Voici le squelette d'un algorithme :

```
ALGORITHME NomDeLAlgorithme
VARIABLES
    variable1 : Type
    variable2 : Type
    // Déclaration des variables utilisées
DEBUT
    // Suite séquentielle d'instructions (de haut en bas)
FIN
```



La variable et l'affectation

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique



C’est quoi une variable ?

Une **variable** est un **espace d'adressage nommé** dans la mémoire vive (RAM) destiné à **stocker** une valeur mutable au cours de l’exécution du programme.

Elle est définie par **4 caractéristiques** immuables durant son cycle de vie :

1. **L'identificateur (Nom)** : Symbole alphanumérique unique (prix_total, age_utilisateur).
2. **Le type** : Domaine de définition mathématique de la donnée.
3. **La valeur** : État binaire instantané contenu à l'adresse mémoire.
4. **L'adresse mémoire** : Emplacement physique/virtuel dans l’espace d'adressage.

Tableau des types primitifs

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Type	Domaine de définition	Exemples
Entier (Z)	Sous-ensemble fini des nombres relatifs.	-5, 0, 42
Réel (R)	Approximations des réels (virgule flottante).	3.14, -0.001
Chaîne de caractères	Séquence ordonnée de caractères (entre guillemets).	"Admin", "Bonjour"
Booléen (B)	Algèbre de Boole : (VRAI, FAUX ou 1, 0).	VRAI, FAUX

L'opération d'affectation

C’est quoi une opération d’affectation ?

En mathématiques, le signe = représente une relation d'**égalité symétrique**.

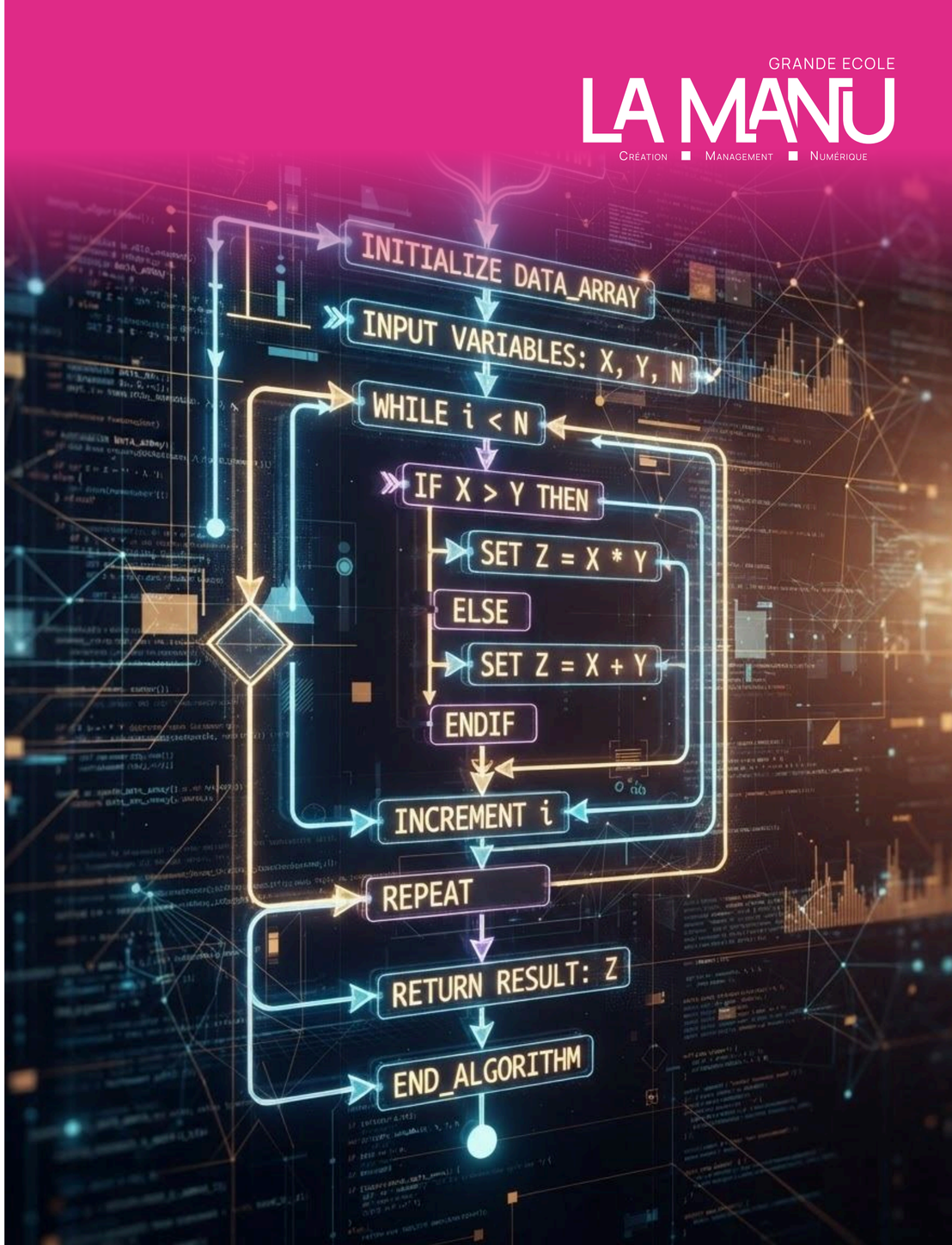
Si je dis **A = B**, c'est exactement la même chose que **B = A**.

En informatique, l'affectation n'est **pas une égalité**.

C'est un ordre, un **transfert d'information** asymétrique qui s'exécute dans le temps.

C'est pour cela qu'en **pseudo-code**, on bannit le signe = pour l'affectation et qu'on utilise la flèche ←

Variable à gauche ← Evaluation puis transfert **Expression (Valeur)**

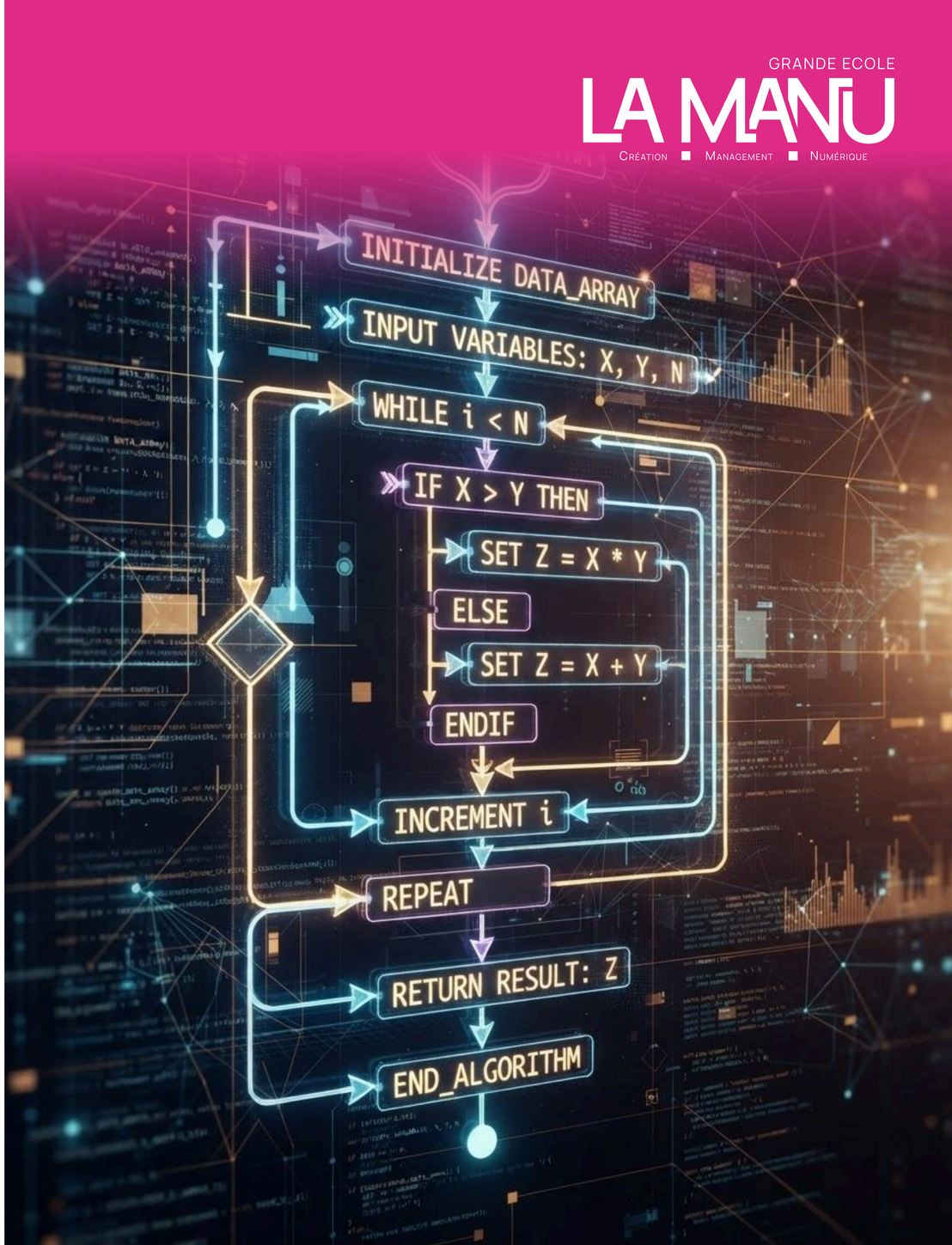


L'opération d'affectation

Variable à gauche ← Evaluation puis transfert **Expression (Valeur)**

Les 2 Étapes fondamentales

1. **Étape 1 - Évaluation (À droite)** : L'ordinateur calcule d'abord la valeur globale de l'expression située à droite du symbole. Tant que ce calcul n'est pas fini, la variable à gauche n'est pas modifiée.
2. **Étape 2 - Écriture / Transfert (À gauche)** : Le résultat de l'évaluation est injecté à l'adresse mémoire désignée par l'identificateur situé à gauche. L'ancienne valeur contenue dans cette variable est immédiatement écrasée et définitivement perdue.



Démonstration des 3 Règles d'Or

GRANDE ECOLE LA MANU
Création ■ Management ■ Numérique

Règle 1
À gauche, uniquement une variable :

❌ 10 ← a : Impossible : 10 est une constante, pas une adresse mémoire !

❌ a + 2 ← b : Impossible : on ne peut pas affecter à une opération

✅ a ← 10

Règle 2
Le typage doit être respecté :

Si “compteur” est déclaré comme **Entier**, on ne peut pas lui faire :

compteur ← ”Bonjour”

Règle 3
L'écrasement est irréversible:

Si une valeur n'est pas sauvegardée dans une autre **variable** avant d'être **écrasée**, elle est **perdue** pour toujours dans la mémoire RAM.

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

La trace mémoire

Le pas à pas de l'algo

Exercice intéractif

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1   Ligne 1 : x = 5
2   Ligne 2 : y = 10
3   Ligne 3 : z = x
4   Ligne 4 : x = y
5   Ligne 5 : y = z
6   Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	?	?	?
2	y = 10	?	?	?
3	z = x	?	?	?
4	x = y	?	?	?
5	y = z	?	?	?
6	z = x + y	?	?	?

Exercice intéractif

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1     Ligne 1 : x = 5
2     Ligne 2 : y = 10
3     Ligne 3 : z = x
4     Ligne 4 : x = y
5     Ligne 5 : y = z
6     Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	5	Non défini	Non défini
2	y = 10	?	?	?
3	z = x	?	?	?
4	x = y	?	?	?
5	y = z	?	?	?
6	z = x + y	?	?	?

Exercice intéractif

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1     Ligne 1 : x = 5
2     Ligne 2 : y = 10
3     Ligne 3 : z = x
4     Ligne 4 : x = y
5     Ligne 5 : y = z
6     Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	5	Non défini	Non défini
2	y = 10	5	10	Non défini
3	z = x	?	?	?
4	x = y	?	?	?
5	y = z	?	?	?
6	z = x + y	?	?	?

Exercice intéractif

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1     Ligne 1 : x = 5
2     Ligne 2 : y = 10
3     Ligne 3 : z = x
4     Ligne 4 : x = y
5     Ligne 5 : y = z
6     Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	5	Non défini	Non défini
2	y = 10	5	10	Non défini
3	z = x	5	10	5
4	x = y	?	?	?
5	y = z	?	?	?
6	z = x + y	?	?	?

Exercice intéractif

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1     Ligne 1 : x = 5
2     Ligne 2 : y = 10
3     Ligne 3 : z = x
4     Ligne 4 : x = y
5     Ligne 5 : y = z
6     Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	5	Non défini	Non défini
2	y = 10	5	10	Non défini
3	z = x	5	10	5
4	x = y	10	10	5
5	y = z	?	?	?
6	z = x + y	?	?	?

Exercice intéractif

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1     Ligne 1 : x = 5
2     Ligne 2 : y = 10
3     Ligne 3 : z = x
4     Ligne 4 : x = y
5     Ligne 5 : y = z
6     Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	5	Non défini	Non défini
2	y = 10	5	10	Non défini
3	z = x	5	10	5
4	x = y	10	10	5
5	y = z	10	5	5
6	z = x + y	?	?	?

Exercice intéractif

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

Consigne : Suivez le code ligne par ligne.
- Pour chaque ligne indiquer la valeur de x ,y et z

```
1     Ligne 1 : x = 5
2     Ligne 2 : y = 10
3     Ligne 3 : z = x
4     Ligne 4 : x = y
5     Ligne 5 : y = z
6     Ligne 6 : z = x + y
```

Ligne	Code	x	y	z
1	x = 5	5	Non défini	Non défini
2	y = 10	5	10	Non défini
3	z = x	5	10	5
4	x = y	10	10	5
5	y = z	10	5	5
6	z = x + y	10	5	15

Les conditions

Le concept de “Control Flow”

La rupture de la séquentialité

Jusqu'à présent, nos algorithmes s'exécutaient de manière purement linéaire ou séquentielle :

De la première à la dernière ligne, sans jamais dévier

L'ordinateur a besoin de **structures de contrôle** pour prendre des décisions.

Les **structures de contrôle** conditionnelles agissent comme des aiguillages sur une voie ferrée : en fonction du résultat d'un test, le programme empruntera :

- un chemin **A**
- un chemin **B**

L’expression Booléenne

Pour qu'une machine prenne une **décision**, il faut lui poser une question **fermée**.

En informatique, on appelle cela une **expression booléenne** (ou **condition**).

Une condition ne peut avoir que deux états (Type Booléen) :

- **VRAI** (True)
- **FAUX** (False)

Les opérateurs relationnels

Pour construire ces **conditions**, on compare des variables ou des valeurs avec des opérateurs :

- `==` : Égal à
- `≠` (ou `<>`, ou `!=`) : Différent de
- `<` : Strictement inférieur à
- `>` : Strictement supérieur à
- `<=` : Inférieur ou égal à
- `>=` : Supérieur ou égal à

Exemples d'évaluations :

- Si **age = 20**, alors la condition **age >= 18** est **VRAIE**.
- Si **solde = 10**, alors la condition **solde > 20** est **FAUSSE**.

Les opérateurs relationnels

Pour construire ces **conditions**, on compare des variables ou des valeurs avec des opérateurs :

- `==` : Égal à
- `≠` (ou `<>`, ou `!=`)
- `<` : Strictement in
- `>` : Strictement s
- `<=` : Inférieur ou
- `>=` : Supérieur ou égal à

Affectation vs Comparaison

Affectation (stockage)
variable ← 10
ou
variable = 10
“Met 10 dans variable”

VS

Comparaison
variable == 10
“Est-ce que variable vaut 10 ?”

Exemples d'évaluations :

- Si **age = 20**, alors la condition **age >= 18** est **VRAIE**.
- Si **solde = 10**, alors la condition **solde > 20** est **FAUSSE**.

Les structures conditionnelles

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

La condition simple : SI ... ALORS :

On l'utilise quand une action ne doit s'exécuter que si une condition est remplie.

Si la condition est fausse, on ne fait rien et on continue la suite de l'algorithme.

Syntaxe pseudo-code :

```
SI condition ALORS
    Instruction(s) à exécuter si VRAI
FIN SI
```

Exemple concret :

```
SI temperature < 0 ALORS
    Afficher("Alerte Gel : Couvrez les plantes !")
FIN SI
```

Les structures conditionnelles

GRANDE ECOLE
LA MANU
Création ■ Management ■ Numérique

L'alternative : SI ... ALORS ... SINON

On l'utilise lorsqu'il y a **deux choix** mutuellement exclusifs.

Le programme est forcé de choisir **l'une des deux voies**, mais il ne peut jamais exécuter les deux à la fois.

Syntaxe pseudo-code :

```
SI condition ALORS
    Instruction(s) si VRAI
SINON
    Instruction(s) si FAUX
FIN SI
```

Exemple concret :

```
SI age >= 18 ALORS
    Afficher("Accès autorisé : Vous êtes majeur.")
SINON
    Afficher("Accès refusé : Vous êtes mineur.")
FIN SI
```

Les structures conditionnelles

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Les conditions imbriquées / multiples : SINON SI

Quand il y a plus de 2 issues possibles, on teste les conditions en cascade, de haut en bas.

Dès qu'une condition est validée (VRAI), le bloc associé s'exécute et tout le reste du bloc conditionnel est ignoré.

Syntaxe pseudo-code :

```
SI condition_1 ALORS
    Action 1
SINON SI condition_2 ALORS
    Action 2
SINON SI condition_3 ALORS
    Action 3
SINON
    Action par défaut (si aucune condition n'est vraie)
FIN SI
```

Exemple concret :

```
SI note >= 10 ALORS
    Afficher("Mention Passable")
SINON SI note >= 16 ALORS
    Afficher("Mention Très Bien")
FIN SI
```

Pour **note = 18**, la première condition **18 >= 10** est **VRAIE**.

Le programme affiche donc **"Mention Passable"** et quitte immédiatement le bloc conditionnel !

Le test note >= 16 n'est même jamais évalué.

Les structures conditionnelles

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Les conditions imbriquées / multiples : SINON SI

Quand il y a plus de 2 issues possibles, on teste les conditions en cascade, de haut en bas.

Dès qu'une condition est validée (VRAI), le bloc associé s'exécute et tout le reste du bloc conditionnel est ignoré.

Règle d'or

L'ordre des conditions est capital.
Dans un intervalle de valeurs, testez toujours du cas le plus restrictif au plus général (du plus grand au plus petit)

Syntaxe pseudo-code :

```
SI condition_1 ALORS
    Action 1
SINON SI condition_2 ALORS
    Action 2
SINON SI condition_3 ALORS
    Action 3
SINON
    Action par défaut (si aucune condition n'est vraie)
FIN SI
```

Exemple concret :

```
SI note >= 10 ALORS
    Afficher("Mention Passable")
SINON SI note >= 16 ALORS
    Afficher("Mention Très Bien")
FIN SI
```

Pour **note = 18**, la première condition **18 >= 10** est **VRAIE**.

Le programme affiche donc **"Mention Passable"** et quitte immédiatement le bloc conditionnel !

Le test note >= 16 n'est même jamais évalué.

Les structures conditionnelles

GRANDE ECOLE
LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Les conditions imbriquées / multiples : SINON SI

Quand il y a **plus de 2** issues possibles, on teste les **conditions en cascade**, de haut en bas.

Dès qu'une condition est **validée** (VRAI), le bloc associé s'exécute et tout le reste du bloc conditionnel est **ignoré**.

Correctif

```
1   SI note >= 16 ALORS
2       Afficher("Mention Très Bien")
3   SINON SI note >= 14 ALORS
4       Afficher("Mention Bien")
5   SINON SI note >= 12 ALORS
6       Afficher("Mention Assez Bien")
7   SINON SI note >= 10 ALORS
8       Afficher("Passable")
9   SINON
10      Afficher("Échec")
11  FIN SI
```

Les structures conditionnelles

GRANDE ECOLE LA MANU
CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Le choix multiple : SELON ... CAS (Switch/Case)

Lorsque l'on doit tester l'égalité d'**une seule et même variable** par rapport à plusieurs valeurs précises, enchaîner des **SI ... SINON SI** devient lourd à lire.

On utilise alors la structure **SELON**.

Syntaxe pseudo-code :

```
SELON variable FAIRE
    CAS valeur_1 :
        Action 1
    CAS valeur_2 :
        Action 2
    AUTREMENT :
        Action par défaut
FIN SELON
```

Exemple (Menu interactif) :

```
Afficher("--- MENU PRINCIPAL ---")
Afficher("1 : Jouer | 2 : Options | 3 : Quitter")
Saisir(choix)

SELON choix FAIRE
    CAS 1 :
        Afficher("Lancement de la partie...")
    CAS 2 :
        Afficher("Ouverture des configurations...")
    CAS 3 :
        Afficher("Fermeture du jeu.")
    AUTREMENT :
        Afficher("Erreur : Choix invalide.")
FIN SELON
```

Les opérateurs logiques

Souvent, un événement dépend de plusieurs **critères simultanés**.

Les opérateurs logiques permettent de **lier** plusieurs expressions booléennes entre elles.

L'opérateur ET (AND) :

Les deux conditions doivent être vraies **en même temps**.

- Exemple : SI (age >= 18) **ET** (possede_permis = VRAI) ALORS ...
- Si l'une des deux est fausse, le résultat global est FAUX.

L'opérateur OU (OR) :

Au moins l'une des conditions doit être vraie.

- Exemple : SI (jour_semaine = "Samedi") **OU** (jour_semaine = "Dimanche") ALORS ...
- Le résultat n'est FAUX que si toutes les conditions sont fausses.

Les opérateurs logiques

L'opérateur NON (NOT) :

Il inverse la valeur de la condition (le VRAI devient FAUX, et inversement).

- Exemple : SI NON (mot_de_passe == "secret123") ALORS Afficher("Erreur")

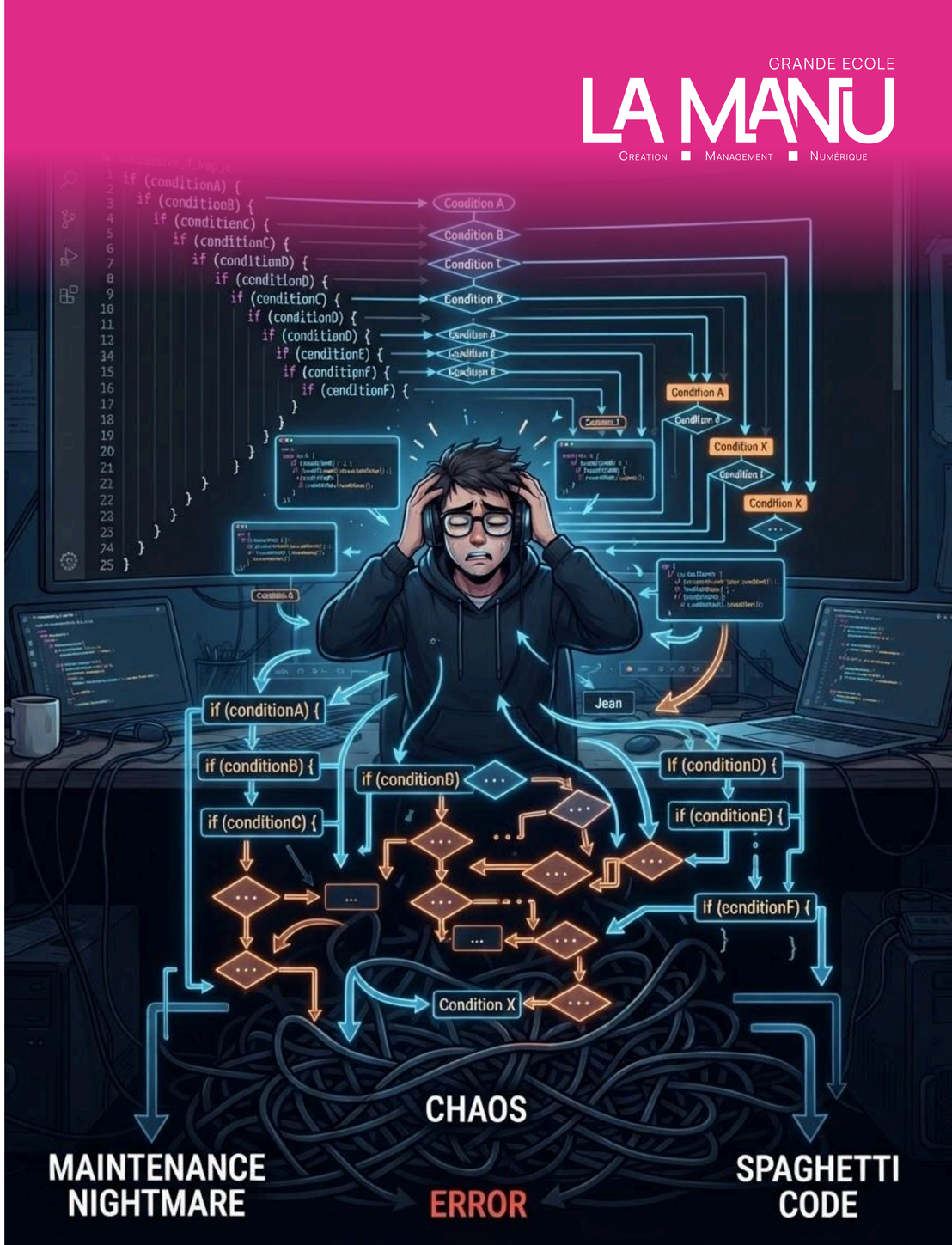
Bonnes pratiques et pièges classiques

Pièges Classiques & Refactoring

Piège #1 : Les SI successifs au lieu d'une alternative

```
1   // ❌ MAUVAIS CODE (Inefficace)
2   SI age < 18 ALORS tarif <- 5 FIN SI
3   SI age >= 18 ALORS tarif <- 10 FIN SI
4   // Le processeur effectue 2 tests inutilement.
```

```
1   // ✅ CODE REFACTORISÉ
2   SI age < 18 ALORS
3       tarif <- 5
4   SINON
5       tarif <- 10
6   FIN SI
```



Pièges Classiques & Refactoring

Piège #2 : Les trous dans les conditions (Oubli des cas limites)

```
// ❌ MAUVAIS CODE
SI nombre < 10 ALORS Afficher("Petit") FIN SI
SI nombre > 10 ALORS Afficher("Grand") FIN SI
```

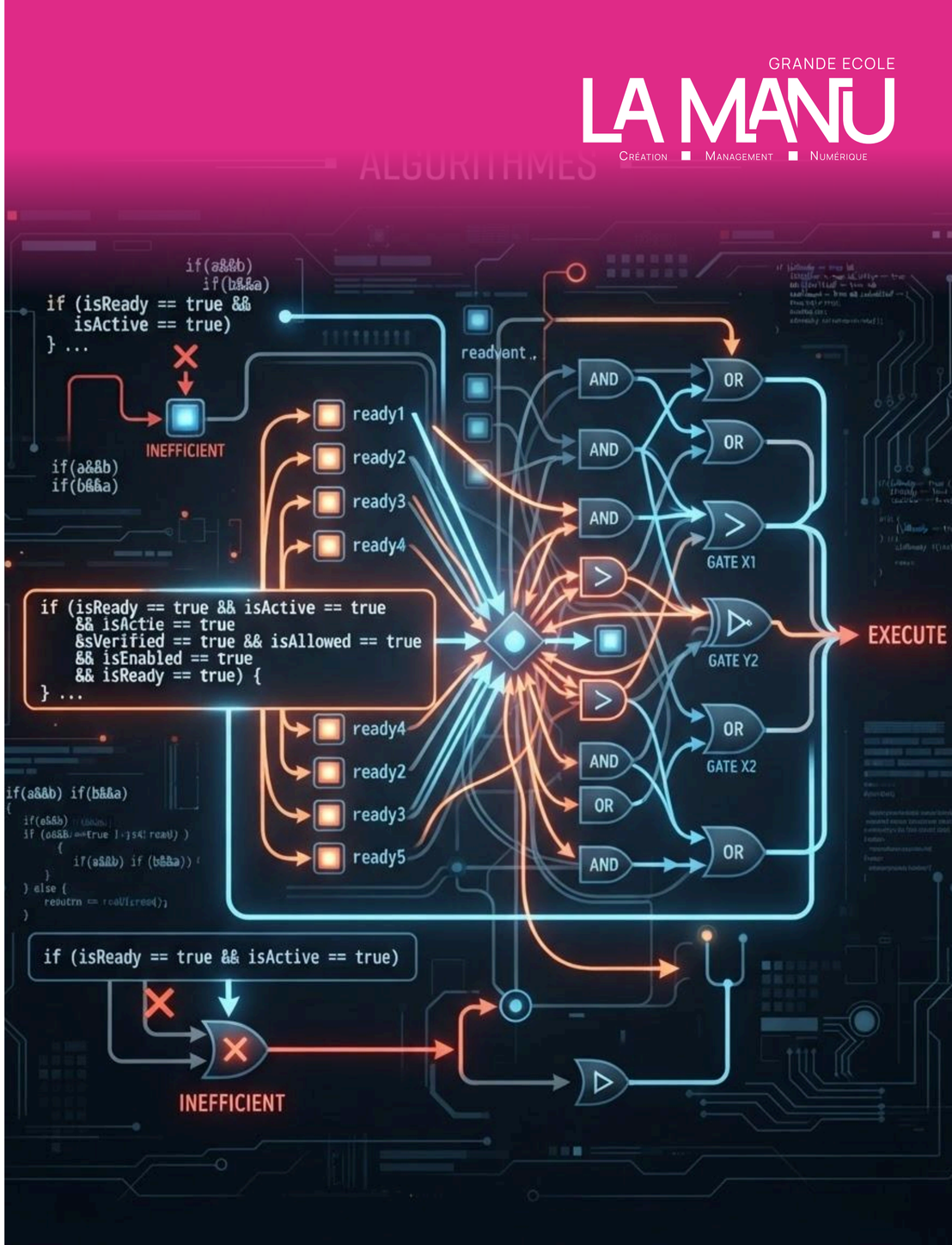
```
// ✅ CODE REFACTORISÉ
SI nombre <= 10 ALORS
    Afficher("Petit ou Égal à 10")
SINON
    Afficher("Grand")
FIN SI
```

Pièges Classiques & Refactoring

Piège #3 : La redondance de booléens

```
// ❌ MAUVAIS CODE (Lourd)
SI est_connecte == VRAI ALORS ... FIN SI
```

```
// ✅ CODE REFACTORISÉ (Élégant)
SI est_connecte ALORS ... FIN SI
SI NON est_connecte ALORS ... FIN SI
```



A VOUS DE JOUER

1er TD